INFORME TÉCNICO

IMPLEMENTACIÓN DEL MODELO FÍSICO DE LA RED SOCIAL ESTUDIANTIL PASCUALINA

Administración y Diseño de Bases de Datos
Ingeniería de Software
Institución Universitaria Pascual Bravo

Integrantes:
Ivan Bolaños
Deimar Copete

Octubre de 2026

1. Información general

Proyecto: Implementación del modelo físico en un SGBD relacional para la Red Social Estudiantil Pascualina

Asignatura: Administración y Diseño de Bases de Datos

Programa académico: Ingeniería de Software

Institución: Institución Universitaria Pascual Bravo

Integrantes: Ivan Bolaños y Deimar Copete

Fecha de entrega: Octubre de 2026

2. Identificación y justificación del SGBD

Se seleccionó PostgreSQL como sistema gestor de bases de datos relacional y pgAdmin 4 como herramienta gráfica de administración. PostgreSQL permite definir claves primarias y foráneas, restricciones CHECK, valores predeterminados y acciones referenciales para mantener la integridad de los datos.

La red social necesita relacionar estudiantes, intereses, conexiones, grupos, publicaciones, eventos y asistentes. Las claves foráneas y las transacciones ayudan a evitar referencias a registros inexistentes y a conservar la consistencia de la información.

3. Transformación del modelo lógico al modelo físico

El modelo se implementa mediante nueve tablas: estudiantes, intereses, estudiante_intereses, conexiones, grupos, miembros_grupo, publicaciones, eventos y asistente_evento. Las relaciones muchos a muchos se resuelven con tablas asociativas y claves primarias compuestas.

Se usa SERIAL para identificadores autoincrementales, INT para referencias numéricas, VARCHAR para textos de longitud definida, TEXT para contenido variable, BOOLEAN para valores lógicos y TIMESTAMP para fechas y horas. Los campos temporales que lo requieren utilizan CURRENT_TIMESTAMP como valor predeterminado.

Se incorporan habilidades al perfil del estudiante, categoría a los grupos y la tabla asistente_evento para registrar la participación de estudiantes en eventos y su confirmación de asistencia.

4. Inventario de tablas

estudiantes: Guarda el perfil del estudiante: estudiante_id SERIAL PK; nombre_completo VARCHAR(120) NOT NULL; correo_institucional VARCHAR(100) NOT NULL UNIQUE; carrera VARCHAR(100) NOT NULL; semestre INT NOT NULL con CHECK entre 1 y 12; biografia TEXT; habilidades TEXT; fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP.

intereses: Catálogo de intereses: interes_id SERIAL PK; nombre VARCHAR(50) NOT NULL UNIQUE; categoria VARCHAR(50) NOT NULL.

estudiante_intereses: Resuelve la relación muchos a muchos entre estudiantes e intereses. estudiante_id e interes_id forman una clave primaria compuesta y son claves foráneas.

conexiones: Representa el seguimiento entre estudiantes y mentorías. seguidor_id y seguido_id forman la clave primaria compuesta; incluye es_mentoria BOOLEAN y fecha_conexion TIMESTAMP. CHECK (seguidor_id <> seguido_id) impide el auto seguimiento.

grupos: Registra grupos de estudio y clubes: grupo_id SERIAL PK; nombre_grupo VARCHAR(100) NOT NULL; descripcion TEXT; categoria VARCHAR(50); creador_id INT NOT NULL como FK; fecha_creacion TIMESTAMP.

miembros_grupo: Relaciona estudiantes con grupos. estudiante_id y grupo_id forman la clave primaria compuesta; también incluye rol y fecha_union.

publicaciones: Guarda publicaciones: publicacion_id SERIAL PK; estudiante_id INT NOT NULL; grupo_id opcional; contenido TEXT NOT NULL; tipo_publicacion VARCHAR(30); fecha_publicacion TIMESTAMP.

eventos: Registra actividades: evento_id SERIAL PK; titulo VARCHAR(150) NOT NULL; descripcion TEXT; fecha_evento TIMESTAMP NOT NULL; lugar VARCHAR(150) NOT NULL; organizador_id INT NOT NULL; grupo_id opcional.

asistente_evento: Resuelve la relación muchos a muchos entre estudiantes y eventos. estudiante_id y evento_id forman la clave primaria compuesta; confirmacion_asistencia BOOLEAN permite indicar si la asistencia está confirmada.

5. Relaciones y acciones referenciales

ON DELETE CASCADE: Se aplica a las tablas asociativas y a los registros dependientes del estudiante, grupo, interés o evento. Al eliminar el registro padre, se eliminan los registros dependientes correspondientes.

ON DELETE SET NULL: En publicaciones.grupo_id y eventos.grupo_id, si se elimina un grupo, la publicación o el evento se conserva y su referencia al grupo queda en NULL.

ON UPDATE CASCADE: Propaga cambios de las claves referenciadas a los campos relacionados.

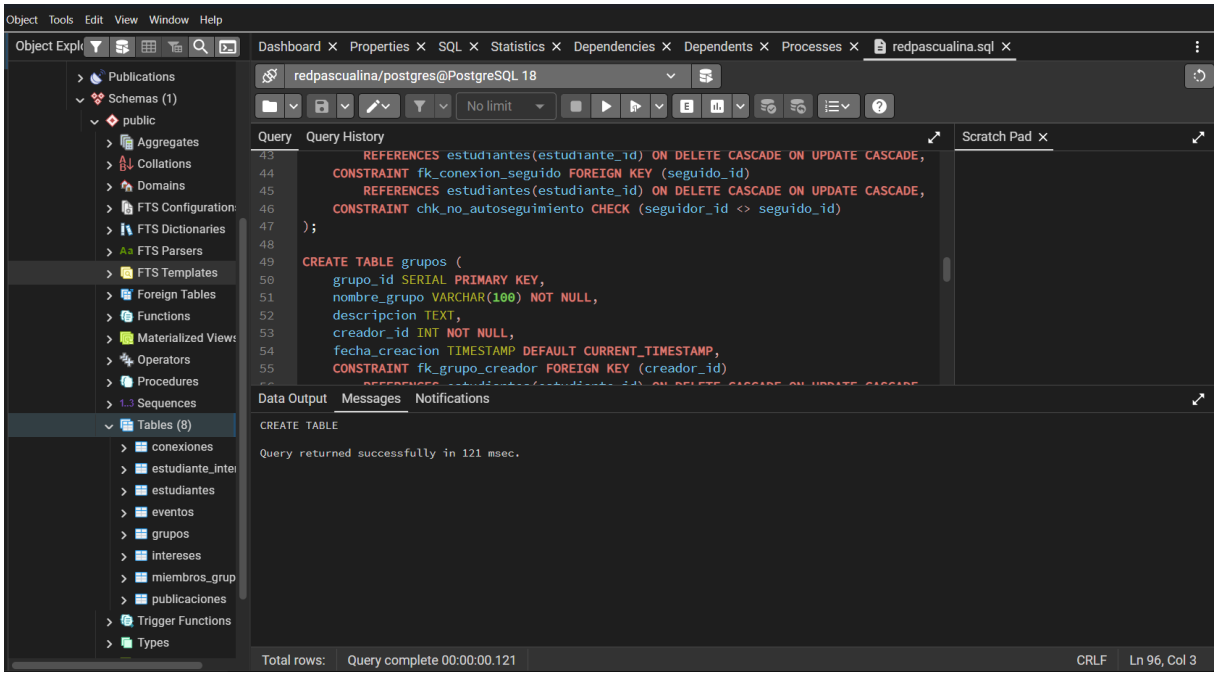
Restricción adicional: La tabla conexiones contiene CHECK (seguidor_id <> seguido_id), que impide que un estudiante se siga a sí mismo.

6. Ejecución del script

El código SQL completo se incluye en el anexo. La captura de pgAdmin 4 suministrada evidencia que las tablas del script original se crearon correctamente. La versión corregida añade la tabla asistente_evento y los campos habilidades y categoria; para que la captura evidencie también esos cambios, se debe volver a ejecutar el script corregido en PostgreSQL y tomar una nueva captura.

7. Evidencia de implementación en PostgreSQL

Captura suministrada de pgAdmin 4 donde se observa la ejecución correcta del script original y la lista de tablas creadas.


Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Processes redpascualina.sql
redpascualina/postgres@PostgreSQL 18
No limit
Query Query History
Scratch Pad
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuration
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (8)
conexiones
estudiante_inter
estudiantes
eventos
grupos
intereses
miembros_grup
publicaciones
Trigger Functions
Types
REFERENCES estudiantes(estudiante_id) ON DELETE CASCADE ON UPDATE CASCADE,
CONSTRAINT fk_conexion_seguido FOREIGN KEY (seguido_id)
REFERENCES estudiantes(estudiante_id) ON DELETE CASCADE ON UPDATE CASCADE,
CONSTRAINT chk_no_autoseguimiento CHECK (seguidor_id <> seguido_id)
);
CREATE TABLE grupos (
grupo_id SERIAL PRIMARY KEY,
nombre_grupo VARCHAR(100) NOT NULL,
descripcion TEXT,
creador_id INT NOT NULL,
fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_grupo_creador FOREIGN KEY (creador_id)
Data Output Messages Notifications
CREATE TABLE
Query returned successfully in 121 msec.
Total rows:
Query complete 00:00:00.121
CRLF
Ln 96, Col 3

Anexo A. Código SQL completo

Nota: el script elimina primero las tablas existentes con esos nombres. Ejecútalo solo en una base de datos de pruebas o con respaldo previo.

```
-- ============================================================
-- PROYECTO: Red Social Estudiantil Pascualina
-- SGBD: PostgreSQL
-- ============================================================

-- ADVERTENCIA: este script elimina las tablas existentes con estos nombres.
-- Ejecutarlo solo en una base de datos de pruebas o con respaldo previo.
DROP TABLE IF EXISTS asistente_evento CASCADE;
DROP TABLE IF EXISTS eventos CASCADE;
DROP TABLE IF EXISTS publicaciones CASCADE;
DROP TABLE IF EXISTS miembros_grupo CASCADE;
DROP TABLE IF EXISTS grupos CASCADE;
DROP TABLE IF EXISTS conexiones CASCADE;
DROP TABLE IF EXISTS estudiante_intereses CASCADE;
DROP TABLE IF EXISTS intereses CASCADE;
DROP TABLE IF EXISTS estudiantes CASCADE;

CREATE TABLE estudiantes (
    estudiante_id SERIAL PRIMARY KEY,
    nombre_completo VARCHAR(120) NOT NULL,
    correo_institucional VARCHAR(100) NOT NULL UNIQUE,
    carrera VARCHAR(100) NOT NULL,
    semestre INT NOT NULL CHECK (semestre > 0 AND semestre <= 12),
    biografia TEXT,
    habilidades TEXT,
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE intereses (
    interes_id SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE,
    categoria VARCHAR(50) NOT NULL
);

CREATE TABLE estudiante_intereses (
    estudiante_id INT NOT NULL,
    interes_id INT NOT NULL,
    PRIMARY KEY (estudiante_id, interes_id),
    CONSTRAINT fk_est_int_estudiante FOREIGN KEY (estudiante_id)
        REFERENCES estudiantes(estudiante_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_est_int_interes FOREIGN KEY (interes_id)
        REFERENCES intereses(interes_id) ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE conexiones (
    seguidor_id INT NOT NULL,
    seguido_id INT NOT NULL,
    es_mentoria BOOLEAN DEFAULT FALSE,
    fecha_conexion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (seguidor_id, seguido_id),
    CONSTRAINT fk_conexion_seguidor FOREIGN KEY (seguidor_id)
        REFERENCES estudiantes(estudiante_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_conexion_seguido FOREIGN KEY (seguido_id)
        REFERENCES estudiantes(estudiante_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT chk_no_autoseguimiento CHECK (seguidor_id <> seguido_id)
);

CREATE TABLE grupos (
    grupo_id SERIAL PRIMARY KEY,
    nombre_grupo VARCHAR(100) NOT NULL,
    descripcion TEXT,
    categoria VARCHAR(50),
    creador_id INT NOT NULL,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_grupo_creador FOREIGN KEY (creador_id)
        REFERENCES estudiantes(estudiante_id) ON DELETE CASCADE ON UPDATE CASCADE
```

```
);

CREATE TABLE miembros_grupo (
    estudiante_id INT NOT NULL,
    grupo_id INT NOT NULL,
    rol VARCHAR(30) DEFAULT 'Miembro',
    fecha_union TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (estudiante_id, grupo_id),
    CONSTRAINT fk_miembro_estudiante FOREIGN KEY (estudiante_id)
        REFERENCES estudiantes(estudiante_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_miembro_grupo FOREIGN KEY (grupo_id)
        REFERENCES grupos(grupo_id) ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE publicaciones (
    publicacion_id SERIAL PRIMARY KEY,
    estudiante_id INT NOT NULL,
    grupo_id INT,
    contenido TEXT NOT NULL,
    tipo_publicacion VARCHAR(30) DEFAULT 'General',
    fecha_publicacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_pub_estudiante FOREIGN KEY (estudiante_id)
        REFERENCES estudiantes(estudiante_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_pub_grupo FOREIGN KEY (grupo_id)
        REFERENCES grupos(grupo_id) ON DELETE SET NULL ON UPDATE CASCADE
);

CREATE TABLE eventos (
    evento_id SERIAL PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    descripcion TEXT,
    fecha_evento TIMESTAMP NOT NULL,
    lugar VARCHAR(150) NOT NULL,
    organizador_id INT NOT NULL,
    grupo_id INT,
    CONSTRAINT fk_evento_organizador FOREIGN KEY (organizador_id)
        REFERENCES estudiantes(estudiante_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_evento_grupo FOREIGN KEY (grupo_id)
        REFERENCES grupos(grupo_id) ON DELETE SET NULL ON UPDATE CASCADE
);

CREATE TABLE asistente_evento (
    estudiante_id INT NOT NULL,
    evento_id INT NOT NULL,
    confirmacion_asistencia BOOLEAN DEFAULT FALSE,
    PRIMARY KEY (estudiante_id, evento_id),
    CONSTRAINT fk_asistente_estudiante FOREIGN KEY (estudiante_id)
        REFERENCES estudiantes(estudiante_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_asistente_evento FOREIGN KEY (evento_id)
        REFERENCES eventos(evento_id) ON DELETE CASCADE ON UPDATE CASCADE
);
```